Lab VII
(Diseño Secuencial I)

Alex Rafael Soriano Severino
Mecatronica
ITLA
La Caleta, Santo Domingo
20240684@itla.edu.do

En resumen— En este laboratorio se diseñó e implementó un circuito secuencial a partir del diagrama de estados mostrado en clases, empleando los cuatro tipos de flip-flops (JK, RS, T y D) para su análisis teórico, y realizando la implementación práctica con flip-flops tipo D. El objetivo fue comprender el proceso de diseño secuencial completo, desde la elaboración de la tabla de verdad y los mapas de Karnaugh (K-maps), hasta la descripción en Verilog y la verificación mediante simulación RTL.
El diseño final incluyó un LED adicional para representar la frecuencia del sistema, ajustada a menos de 40 Hz, y LEDs indicadores de los diferentes estados de la máquina. El circuito fue desarrollado de acuerdo con las técnicas enseñadas en clase, sin el uso de estructuras *case* o *for*, empleando únicamente bloques *always* sincronizados por reloj y lógica combinacional para las interconexiones entre flip-flops.

Abstract— In this laboratory, a sequential circuit was designed and implemented based on the state diagram shown in classes, using the four types of flip-flops (JK, RS, T and D) for theoretical analysis, and carrying out the practical implementation with flip-flops. flops type D. The objective was to understand the complete sequential design process from the preparation of the truth table and Karnaugh maps (K-maps), to the description in Verilog and verification using RTL simulation.

The final design included an additional LED to represent the system frequency, set to less than 40 Hz, and LEDs indicating the different states of the machine. The circuit was developed according to the techniques taught in class, without the use of case or for structures, using only blocks always synchronized by clock and combinational logic for the interconnections between flip-flops..

***Palabras claves— lógica secuencial, flip-flop D, K-map, Verilog, FPGA, diagrama de estados, diseño digital, RTL*.**

I. INTRODUCCION

El propósito de este laboratorio fue diseñar un sistema secuencial digital a partir de un diagrama de estados predefinido, aplicando los conceptos fundamentales de diseño secuencial y la metodología de conversión de diagramas de estados a ecuaciones lógicas.
En primera instancia, se analizaron los cuatro tipos de flip-flops (JK, RS, T y D) para comprender sus tablas de excitación y su aplicación dentro de una máquina de estados. Posteriormente, se desarrolló el diseño lógico utilizando exclusivamente flip-flops tipo D, determinando sus entradas a partir de las expresiones obtenidas mediante mapas de Karnaugh (K-maps).
El diseño fue implementado tanto a nivel lógico (en papel) como a nivel de descripción hardware mediante el lenguaje Verilog. Finalmente, se realizó la simulación del comportamiento temporal (diagrama de tiempo) y la comprobación del funcionamiento físico mediante LEDs conectados a los estados de salida y un LED adicional que permitió visualizar la frecuencia del reloj (menor a 40 Hz).

II. MARCO TEÓRICO

A. Flip-Flop D

El flip-flop tipo D (Data o Delay) es un biestable que almacena un bit de información. Su salida cambia únicamente en los flancos activos del reloj, copiando el valor presente en su entrada D.

B. Máquina de Estados

El demultiplexor realiza la operación inversa del MUX: una sola entrada es dirigida hacia una de varias salidas posibles, dependiendo del valor de las líneas de selección.
Se emplea en sistemas de distribución de datos, control de buses y decodificación de señales.

Una máquina de estados es un sistema secuencial que cambia de estado en función de las entradas y del reloj. Se compone de tres partes principales:

1. Registro de estado (flip-flops)
2. Lógica de siguiente estado
3. Lógica de salida

El diseño parte del **diagrama de estados**, a partir del cual se obtiene la **tabla de transición**, la **tabla de verdad** y, finalmente, las ecuaciones simplificadas mediante **K-maps**.

C. K-Maps (Mapas de Karnaugh)

Los mapas de Karnaugh permiten simplificar las funciones booleanas que determinan las entradas D de cada flip-flop, reduciendo el número de compuertas necesarias en la implementación.
Cada flip-flop D tiene su propia ecuación de excitación derivada del análisis de la tabla de transición.

D. Diseño Secuencial en Verilog

En Verilog, el diseño secuencial se describe usando bloques *always* sincronizados al flanco del reloj. En este laboratorio, no se permitió el uso de estructuras *case* o *for*, siguiendo una codificación estrictamente estructural basada en ecuaciones combinacionales y registros D.

III. MONTAJE EXPERIMENTAL

Etapa 1: Diseño Lógico y K-Maps.

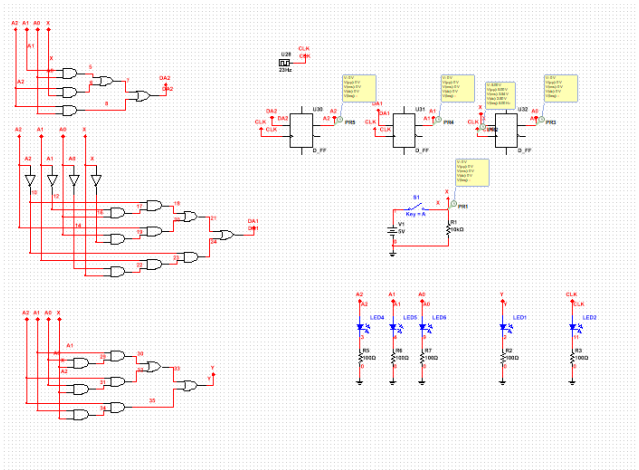
A partir del diagrama de estados proporcionado, se elaboró la tabla de transición de estados, determinando los valores de las entradas D para cada flip-flop en función del estado actual y las entradas. Luego, se aplicaron mapas de Karnaugh para cada flip-flop con el fin de obtener las ecuaciones simplificadas de excitación:

Estas ecuaciones fueron implementadas con compuertas lógicas básicas (AND, OR, NOT) en el diagrama lógico.
Etapa 2: Implementación Práctica con Flip-Flops D
El circuito fue armado usando flip-flops tipo D reales o equivalentes en FPGA, con LEDs conectados a las salidas Q para representar los diferentes estados.
Un LED adicional fue empleado como indicador de la frecuencia del reloj, asegurando que fuera menor de 40 Hz para una visualización clara de las transiciones.
Etapa 3: Implementación en Verilog y Simulación RTL
El código fue escrito en Verilog usando únicamente bloques *always* para la lógica secuencial y combinacional.
Se verificó el comportamiento del circuito mediante simulación RTL, observando el diagrama de tiempo, donde se comprobó que las transiciones entre estados coincidían con el diagrama de estado original.

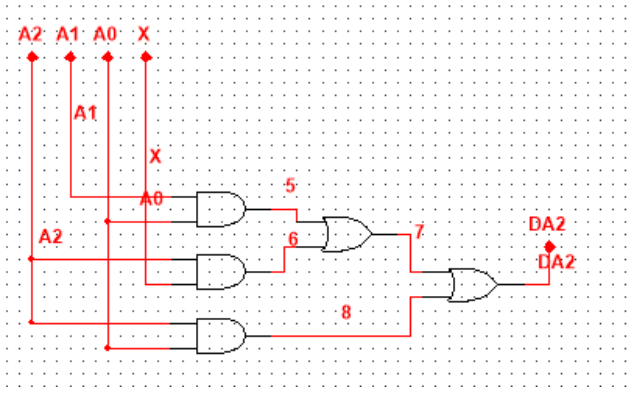
IV. FUNCIONAMIENTO DEL CIRCUITO:

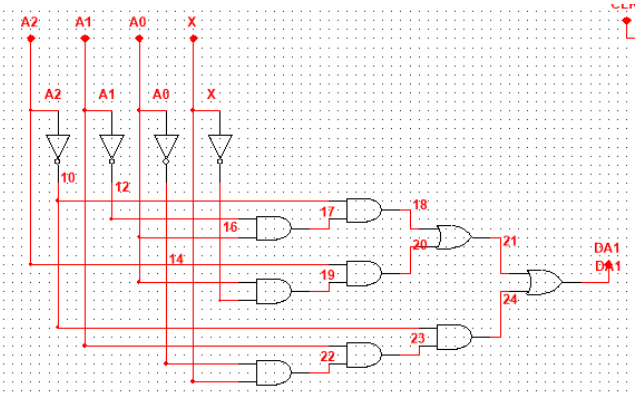
El circuito secuencial diseñado funciona siguiendo el diagrama de estados mostrado en clases, utilizando flip-flops tipo D para almacenar y cambiar de estado con cada pulso del reloj. Cada estado se representa mediante LEDs conectados a las salidas Q, permitiendo observar visualmente las transiciones. La lógica combinacional determina las entradas D a partir del estado actual, asegurando que el sistema avance de forma ordenada y sincronizada. En cada flanco ascendente del reloj, los flip-flops actualizan sus valores y el circuito progresa a la siguiente etapa definida en el diagrama. Un LED adicional, conectado a un divisor de frecuencia menor a 40 Hz, permite visualizar el ritmo de funcionamiento del reloj. Durante las pruebas y la simulación en Verilog, se comprobó que las transiciones coincidían con las del diseño teórico, demostrando que el sistema opera de manera estable, secuencial y completamente sincronizada con la señal de reloj.

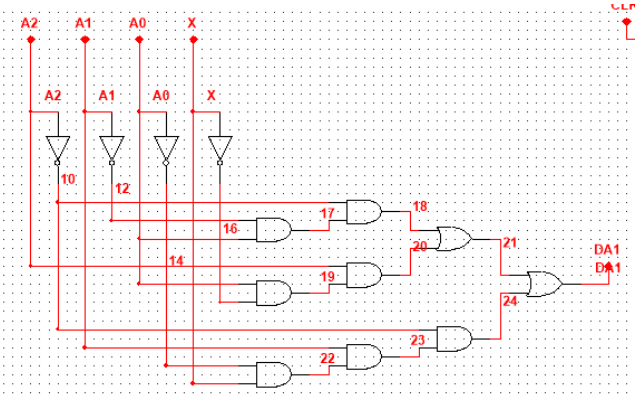
Diseño lógico (MULTISIM):



Para las entradas de los flip flops:







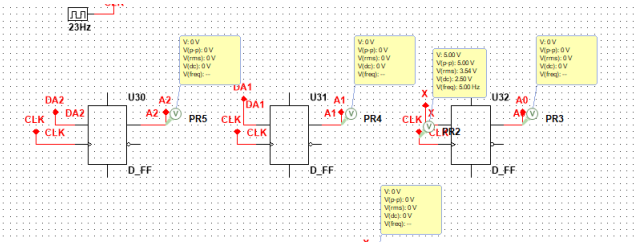


Tabla de estado y de excitación para el flip flop d:

		Presente			X	Futuro			Y	DA2	DA1	DA0
		A2	A1	A0		A2+	A1+	A0+				
a	1	0	0	0	0	0	0	0	0	0	0	0
a	2	0	0	0	1	0	0	1	0	0	0	1
b	3	0	0	1	0	0	1	0	0	0	1	0
b	4	0	0	1	1	0	1	1	0	0	1	1
c	5	0	1	0	0	0	0	0	0	0	0	0
c	6	0	1	0	1	0	1	1	0	0	1	1
d	7	0	1	1	0	1	0	0	0	1	0	0
d	8	0	1	1	1	1	0	1	1	1	0	1
e	9	1	0	0	0	0	0	0	0	0	0	0
e	10	1	0	0	1	1	0	1	1	1	0	1
f	11	1	0	1	0	1	1	0	0	1	1	0
f	12	1	0	1	1	1	0	1	1	1	0	1
g	13	1	1	0	0	0	0	0	0	0	0	0
g	14	1	1	0	1	1	0	1	1	1	0	1
		1	1	1	0	X	X	X	X	X	X	X
		1	1	1	1	X	X	X	X	X	X	X

DA2				
A2-A1/A0-X	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	0	1	X	X
10	0	1	1	1

Y				
A2-A1/A0-X	00	01	11	10
00	0	0	0	0
01	0	0	1	0
11	0	1	X	X
10	0	0	1	0

DA1				
A2-A1/A0-X	00	01	11	10
00	0	0	1	1
01	0	1	0	0
11	0	0	X	X
10	0	0	0	1

DA0				
A2-A1/A0-X	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	1	X	X
10	0	1	1	0

SoP (DA2)
BC + AD + AC

A1A0 + A2X + A2A0

SoP (DA1)
A'B'C + ACD' + A'BC'D
A2'A1'A0' + A2A0X' + A2'A1A0'X

SoP (DA0)
D
X

SoP (Y)
BCD + ACD + ABD
A1A0X + A2A0X + A2A1X

La tabla de estados vendría siendo la mismas para todos los caso lo que va a variar es la tabla o la parte de excitación de la tabla que dependerá del tipo de FF que escojamos. Aquí les mostrare la tabla de excitación de los otros tipos de FF:

FF JK:

	A2+	A1+	A0+	J2	K2	J1	K1	J0	K0
1	0	0	0	0	X	0	X	0	X
2	0	0	1	0	X	0	X	1	X
3	0	1	0	0	X	1	X	X	1
4	0	1	1	0	X	1	X	X	0
5	0	0	0	0	X	X	1	0	X
6	0	1	1	0	X	X	0	1	X
7	1	0	0	1	X	X	1	X	1
8	1	0	1	1	X	X	1	X	0
9	0	0	0	X	1	0	X	0	X
10	1	0	1	1	X	0	X	1	X
11	1	1	0	X	0	1	X	X	1
12	1	0	1	X	0	0	X	X	0
13	0	0	0	X	1	X	1	0	X
14	1	0	1	X	0	X	1	1	X
	X	X	X	X	X	X	X	X	X
	X	X	X	X	X	X	X	X	X

J2				
A2-A1/A0-X	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	X	1	X	X
10	X	X	X	X

K2				
A2-A1/A0-X	00	01	11	10
00	X	X	X	X
01	X	X	X	X
11	1	X	0	0
10	1	0	X	X

J1				
A2-A1/A0-X	00	01	11	10
00	0	0	1	1
01	X	X	X	X
11	0	0	0	1
10	X	X	X	X

K1				
A2-A1/A0-X	00	01	11	10
00	X	X	X	X
01	1	0	1	1
11	X	X	X	X
10	1	1	X	X

J0				
A2-A1/A0-X	00	01	11	10
00	0	1	X	X
01	0	1	X	X
11	0	1	X	X
10	0	1	X	X

K0				
A2-A1/A0-X	00	01	11	10
00	X	X	0	1
01	X	X	0	1
11	X	X	X	X
10	X	X	0	1

FF SR:

	A2+	A1+	A0+	S2	R2	S1	R1	S0	R0
1	0	0	0	0	X	0	X	0	X
2	0	0	1	0	X	0	X	1	0
3	0	1	0	0	X	1	0	0	X
4	0	1	1	0	X	1	0	X	0
5	0	0	0	0	X	0	1	0	X
6	0	1	1	0	X	X	0	1	0
7	1	0	0	1	0	0	1	0	1
8	1	0	1	1	0	0	1	X	0
9	0	0	0	0	1	0	X	0	X
10	1	0	1	X	0	0	X	1	0
11	1	1	0	X	0	1	0	0	1
12	1	0	1	X	0	0	X	X	0
13	0	0	0	0	1	0	1	0	X
14	1	0	1	X	0	0	1	1	0
	X	X	X	X	X	X	X	X	X
	X	X	X	X	X	X	X	X	X

S2				
A2-A1/A0-X	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	0	X	X	X
10	0	X	X	X

R2				
A2-A1/A0-X	00	01	11	10
00	X	X	X	X
01	X	X	0	0
11	1	0	X	X
10	1	0	0	0

S1				
A2-A1/A0-X	00	01	11	10
00	0	0	1	1
01	0	X	0	0
11	0	0	X	X
10	0	0	0	1

R1				
A2-A1/A0-X	00	01	11	10
00	X	X	0	0
01	1	0	1	1
11	1	1	X	X
10	X	X	0	X

S0				
A2-A1/A0-X	00	01	11	10
00	0	1	X	0
01	0	1	X	0
11	0	1	X	X
10	0	1	X	0

R0				
A2-A1/A0-X	00	01	11	10
00	X	0	X	0
01	X	0	0	1
11	X	0	0	1
10	X	0	X	X

FF T:

	A2+	A1+	A0+	T2	T1	T0
1	0	0	0	0	0	0
2	0	0	1	0	0	1
3	0	1	0	0	1	1
4	0	1	1	0	1	0
5	0	0	0	0	1	0
6	0	1	1	0	0	1
7	1	0	0	1	1	1
8	1	0	1	1	1	0
9	0	0	0	1	0	0
10	1	0	1	0	0	1
11	1	1	0	0	1	1
12	1	0	1	0	0	0
13	0	0	0	0	1	0
14	1	0	1	1	1	1
	X	X	X	X	X	X
	X	X	X	X	X	X

T2				
A2-A1/A0-X	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	1	0	0	0
10	0	1	X	X

T1				
A2-A1/A0-X	00	01	11	10
00	0	0	1	1
01	1	0	1	1
11	1	1	X	X
10	0	0	0	1

T0				
A2-A1/A0-X	00	01	11	10
00	0	1	0	1
01	0	1	1	0
11	0	1	X	X
10	0	1	0	1

A continuación, le mostrare captura de los módulos en la programación en Gowin.

Módulos para las entradas de los flip flops:

```
1  //Module Inputs_FF_DA2
2  module INPUTS_FF_DA2(
3      // Inputs and outputs declaration
4      input A2,A1,A0,A,
5      output DA2
6  );
7      //WIRES
8      wire  C1,C2,C3,C4;
9
10     // Declarations
11     and AND1(C1,A1,A0);
12     and AND2(C2,A2,A);
13     and AND3(C3,A2,A0);
14     or OR1(C4,C1,C2);
15     or OR2(DA2,C4,C3);
16 endmodule
17
18
19
20 //Module Inputs_FF_DA1
21 module INPUTS_FF_DA1(
22     // Inputs and outputs declaration
23     input A2,A1,A0,A,
24     output DA1
25 );
26     //WIRES
27     wire  C1,C2,C3,C4,C5,C6,C7,C8,C9,C10,C11,C12;
28
29     // Declarations
30     not N1(C1,A2);
31     not N2(C2,A1);
32     not N3(C3,A0);
33     not N4(C4,A);
34     and AND1(C5,C2,A0);
35     and AND2(C6,A0,C4);
36     and AND3(C7,C3,A);
37     and AND4(C8,C1,C5);
38     and AND5(C9,A2,C6);
39     and AND6(C10,A1,C7);
40     and AND7(C11,C1,C10);
41     or OR1(C12,C8,C9);
42     or OR2(DA1,C12,C11);
43 endmodule
44
45
46
47
48 //Module OUTPUTS_Y
49 module OUTPUTS_Y(
50     // Inputs and outputs declaration
51     input A2,A1,A0,A,
52     output Y
53 );
54     //WIRES
55     wire  C1,C2,C3,C4,C5,C6,C7;
56
57     // Declarations
58     and AND1(C1,A0,A);
59     and AND2(C2,A0,A);
60     and AND3(C3,A1,A);
61     and AND4(C4,A1,C1);
62     and AND5(C5,A2,C2);
63     and AND6(C6,A2,C3);
64     or OR1(C7,C4,C5);
65     or OR2(Y,C7,C6);
66 endmodule
67
```

Modulo del Flip Flop D:

```
//Module FF_D
module FF_D(
    // Inputs and outputs declaration
    input D,CLK,
    output reg Q
);
    always @(posedge CLK)
    begin
        Q <= D;
    end
endmodule
```

Modulo del clock divider:

```
module CLOCK_DIVIDER(
    input clock_in, // input clock on FPGA
    output reg clock_out // output clock after dividing the input clock by divisor
);
reg[27:0] counter=28'd0;
parameter DIVISOR = 28'd100_000_000;
always @(posedge clock_in)
begin
    counter <= counter + 28'd1;
    if(counter>=(DIVISOR-1))
     counter <= 28'd0;
    clock_out <= (counter<DIVISOR/2)?1'b1:1'b0;
end
endmodule
```

Modulo Main con todas las iontanciaciones:

```
//Module Main FF_D
module MAIN_FF_D(
    // Inputs and outputs declaration
    input A,CLOCK_D,
    output L1,L2,L3,L_CLK,L4
);

    //WIRES
    wire  CLO,DA2,A2,DA1,A1,A0;

    CLOCK_DIVIDER CLK_DVD (
        .clock_in(CLOCK_D),
        .clock_out(L_CLK)
    );

    //Instantiates
    INPUTS_FF_DA2 I_FF_DA2 (
        .A2(A2),
        .A1(A1),
        .A0(A0),
        .A(A),

        .DA2(DA2)
    );

    FF_D FF_D_1 (
        .D(DA2),
        .CLK(L_CLK),

        .Q(A2)
    );

    INPUTS_FF_DA1 I_FF_DA1 (
        .A2(A2),
        .A1(A1),
        .A0(A0),
        .A(A),

        .DA1(DA1)
    );

    FF_D FF_D_2 (
        .D(DA1),
        .CLK(L_CLK),

        .Q(A1)
    );

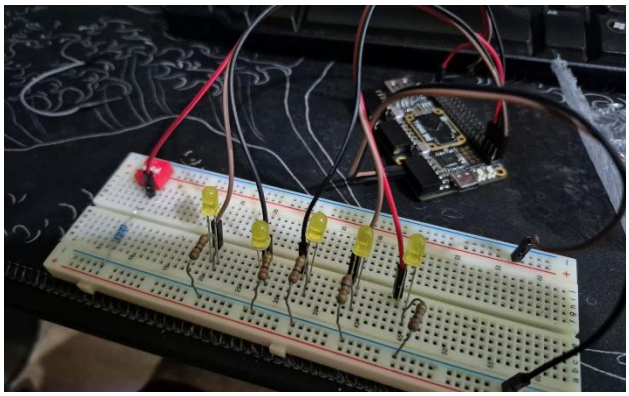
    FF_D FF_D_3 (
        .D(A),
        .CLK(L_CLK),

        .Q(A0)
    );

    OUTPUTS_Y O_FF_Y (
        .A2(A2),
        .A1(A1),
        .A0(A0),
        .A(A),

        .Y(L4)
    );
endmodule
```

V. FOTOS DEL CIRCUITO EN FUNCIONAMIENTO



VI. CONCLUSION

El laboratorio permitió comprender de forma integral el proceso de diseño secuencial, desde el análisis teórico del diagrama de estados hasta la implementación física y simulada de un sistema con flip-flops D.

Se consolidó el conocimiento sobre cómo obtener las ecuaciones de excitación mediante K-maps y cómo implementar máquinas de estados en Verilog utilizando un enfoque estructural.

El uso de LEDs como indicadores de estado facilitó la visualización del comportamiento del circuito, mientras que el LED de frecuencia permitió comprobar el funcionamiento del reloj.

En conclusión, este laboratorio reforzó los conceptos fundamentales del diseño digital secuencial, la simplificación lógica y la correcta aplicación del lenguaje Verilog para describir sistemas de hardware sincronizados por reloj.

VII. REFERENCIAS

https://en.wikipedia.org/wiki/Digital_electronics
https://naylampmechatronics.com/blog/39_resistencias-pull-up-y-pull-down.html
https://es.wikipedia.org/wiki/Sistema_de_numeraci%C3%B3n
https://es.wikipedia.org/wiki/C%C3%B3digo_binario
https://es.slideshare.net/slideshow/sistemas-de-codigos-en-la-electronica-digital/158426000